AI TECHNICAL DEBT PRIORITIZER

Design Changes and Their Justification

What changed from the original idea, what would have gone wrong, and why the change fixes it

Purpose. The original idea for the AI Technical Debt Prioritizer was sound in direction but left several decisions open, and a few of them, taken literally, would have produced results that look good and mean nothing. This document records every change made in the final design (Documents 1 and 2). For each change it states what the original plan said, what problem that would cause, what was changed, and why the change solves the problem or is simply better.

How to read it. Section 1 summarises the original plan. Section 2 is a one-page list of all changes. Sections 3 to 9 give the full case for each change, grouped by area. Section 10 lists what was kept and what was considered and rejected. Section 11 collects likely review and viva questions with short answers.

Labels used. **Fixes a flaw**: the original would have produced wrong or unconvincing results. **Improvement**: the original would work, but this is clearly better. **Scope decision**: a deliberate narrowing so the project can be finished well.

Contents

1. The Original Plan . . . 3

2. Changes at a Glance . . . 4

3. Labels, Leakage and the Dataset . . . 6

4. Learning Objective and Scoring . . . 8

5. Evaluation . . . 10

6. Features . . . 12

7. Explainability and the AI Layer . . . 15

8. Scope . . . 17

9. Engineering and Platform . . . 18

10. What Was Kept, and What Was Rejected . . . 21

10.1 Kept from the original plan . . . 21

10.2 Considered and rejected . . . 21

11. Anticipated Questions and Answers . . . 22

1. The Original Plan

The starting point was a one-page idea. Its key decisions, as written, were:

Aspect	Original plan
Goal	Rank technical-debt issues by how urgently they should be fixed, not just detect code smells
Inputs	code complexity, churn and git history, dependencies, test coverage, bug/fix history, duplication
Output	"Technical Debt Priority Score: 92/100 (Critical)" with top reasons and a recommended fix order
Novelty	context-aware prioritisation: code metrics + repository evolution + dependency impact, prioritised by AI
Dataset	self-built from public GitHub repositories; per file or function: complexity, LOC, dependency count, churn, commit frequency, bug-fix commits, contributors, test coverage, dependents
Labels	"Label technical-debt priority using historical maintenance/bug activity, then train Random Forest/XGBoost"
Extra data	supplement with PROMISE datasets
Stack	Python, Pandas/NumPy, scikit-learn, XGBoost, SHAP; Tree-sitter, GitPython, NetworkX; FastAPI; Next.js, TypeScript, Tailwind, Recharts; PostgreSQL optional

Table 1. Summary of the original idea.

The direction is right and most of the stack is kept. The problems are in what the plan leaves undefined: what exactly the label is, which time period it covers, what the score means, how "fix order" is computed, and how anyone would know the model is better than a simple rule. Those gaps are where a project like this usually fails under review.

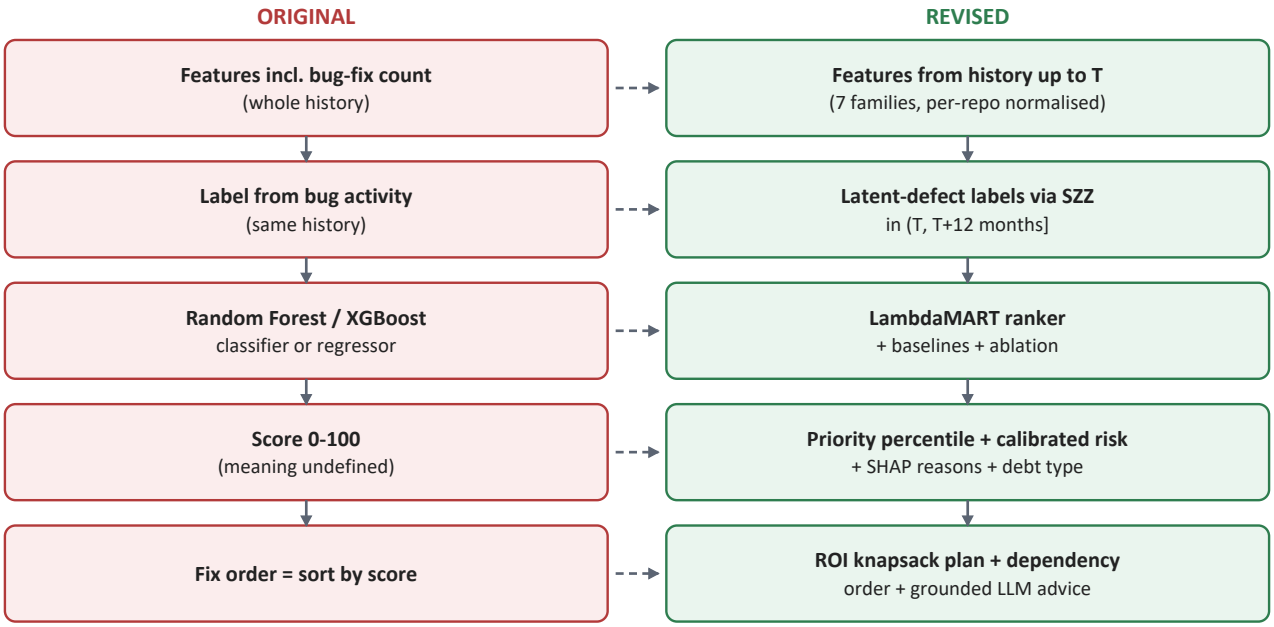


Figure 1. The pipeline before and after the changes. Each dashed arrow corresponds to one or more changes in this document.

2. Changes at a Glance

There are 39 changes: 20 fix a flaw, 17 are improvements and 2 are scope decisions. The three marked **Critical** would each, on their own, invalidate the results or the service: leakage between features and labels (C1), random-split evaluation (C11), and an unprotected service that processes untrusted code (C38).

ID	Change	Type	Severity
C1	Separate the feature window from the label window	Fixes a flaw	Critical
C2	Define bugs with SZZ latent-defect labels	Fixes a flaw	High
C3	Turn "priority" into a defined, graded label	Fixes a flaw	High
C4	Several snapshots per repository, with selection criteria	Improvement	-
C5	Measure the quality of bug-fix detection	Fixes a flaw	Medium
C6	Learning-to-rank (LambdaMART) instead of plain classification	Fixes a flaw	High
C7	Give the 0 to 100 score a defined meaning	Fixes a flaw	High
C8	Compute the fix order with return on effort and a budget	Fixes a flaw	High
C9	Order the plan by dependencies	Improvement	-
C10	Add strong baselines; keep Random Forest as one of them	Fixes a flaw	High
C11	Split by time and by project, never by random row	Fixes a flaw	Critical
C12	Use ranking and effort-aware metrics	Fixes a flaw	High
C13	Prove the novelty claim with an ablation study	Fixes a flaw	High
C14	Report significance and confidence intervals	Improvement	-
C15	Use PROMISE as a sanity check, not training data	Fixes a flaw	Medium
C16	Add co-change coupling	Improvement	-
C17	Ownership features instead of a contributor count	Improvement	-
C18	Recency-weighted churn and trend features	Improvement	-
C19	Graph centrality instead of a simple dependents count	Improvement	-
C20	Static test signals instead of measured coverage	Fixes a flaw	High
C21	Normalise features within each repository	Fixes a flaw	Medium
C22	Add self-admitted technical debt (SATD)	Improvement	-
C23	Specify the duplication algorithm	Improvement	-
C24	Group SHAP values by feature family and turn them into sentences	Fixes a flaw	Medium
C25	Assign a debt type and a remedy	Improvement	-
C26	A grounded LLM advisor for the top files	Improvement	-
C27	A what-if simulator, honestly framed	Improvement	-
C28	Collect feedback and learn from it	Improvement	-
C29	File level first; function level later	Scope decision	-
C30	Python first, Java second	Scope decision	-
C31	Background jobs with progress instead of synchronous requests	Fixes a flaw	High
C32	PostgreSQL required, not optional	Fixes a flaw	Medium

ID	Change	Type	Severity
C33	Incremental analysis with content-addressed caching	**Improvement**	-
C34	One shared core package for training and serving	**Fixes a flaw**	Medium
C35	Model versioning, promotion gates and no pickle files	**Improvement**	-
C36	Bring results into pull requests (GitHub Action and SARIF)	**Improvement**	-
C37	Use a graph library for the dependency view	**Fixes a flaw**	Low
C38	Security hardening for untrusted repositories	**Fixes a flaw**	Critical
C39	Tests, CI/CD and containers from day one	**Improvement**	-

Table 2. All changes.

3. Labels, Leakage and the Dataset

C1 Separate the feature window from the label window	Fixes a flaw | Critical
Original plan	Bug-fix commits are an input feature, and the label is derived from "historical maintenance/bug activity", with no time boundary between the two.
Problem it causes	The model is shown the answer. If the label is "lots of bug activity" and a feature is "number of bug-fix commits" over the same history, the model learns to copy the feature into the label. Accuracy will look excellent (often above 0.9 AUC), but the model has learned nothing about the future; it describes the past. This is target leakage, and it is the first thing a reviewer will look for.
Change	For each snapshot time T, every feature uses only history up to T; the label uses only what happens in (T, T+12 months]. Training snapshots are placed so that no training label window crosses the test snapshot (an embargo). Past bugs stay as a feature because they are known at time T.
Why this solves it	The model now answers a real question, "which files will cause pain next", the same question a user has when they run it. A leakage test (recompute features from a checkout at T and compare) and a label-shuffle test prove the separation in code.
Cost / trade-off	Fewer usable rows (the last 12 months of each repository can only be used for labels), and somewhat lower headline metrics. Those lower numbers are the honest ones.
One-line defence	*"Our features stop at time T and our labels start after it, so the model predicts the future instead of restating the past."*

C2 Define bugs with SZZ latent-defect labels	Fixes a flaw | High
Original plan	"Historical maintenance/bug activity" per file, with no definition of how a bug is attributed to a file.
Problem it causes	A bug-fix commit usually also touches tests, changelogs, documentation and unrelated clean-up. Counting every touched file as buggy floods the labels with false positives. Commit-message keywords add more noise ("fix typo", "prefix"). Noisy labels cap how good any model can be and make evaluation unreliable.
Change	Use the SZZ algorithm: take the lines a fix commit changed, run git blame on its parent, and find the commit that introduced those lines. A file gets a latent defect at T only if a fix after T repairs lines that already existed at T. Bug-fix commits are identified first by issue links with bug labels, then by filtered keywords.
Why this solves it	SZZ attributes each bug to the file and the change where it actually lived, which is the standard method in defect-prediction research. The latent-defect rule matches the meaning of "debt": a problem present in the code now that will cost effort later.
Cost / trade-off	Blame is slow, so it runs offline and is cached. SZZ is not perfect either; the design measures detector precision on 200 hand-labelled commits and reports it.
One-line defence	*"We label a file only when a later fix repairs code that already existed at prediction time, traced line by line with SZZ."*

C3 Turn "priority" into a defined, graded label

Fixes a flaw | High

Original plan	"Label technical-debt priority" without saying what priority is measured as.
Problem it causes	Without a definition, the label can quietly change between experiments, nobody can reproduce it, and an examiner can reasonably say the target is arbitrary. A binary "buggy or not" label also throws away the difference between a file with one minor fix and one with eight.
Change	Maintenance pain P = latent defects + 0.25 x log(1 + churn / repository median churn), mapped to grades 0 to 4 by quantiles within each repository-snapshot. A sensitivity analysis varies the churn weight and the horizon and reports how much the top-10 lists move.
Why this solves it	The label is explicit, reproducible and includes both risk (defects) and cost of upkeep (churn). Graded labels are what ranking models need. The sensitivity analysis answers "what if you had defined it differently?" before anyone asks.
Cost / trade-off	The weight 0.25 is a judgement call; that is why its effect is measured and reported.
One-line defence	*"Priority is defined as future maintenance pain, a written formula, and we show the results hold when the formula's weights change."*

C4 Several snapshots per repository, with selection criteria

Improvement

Original plan	Build a dataset from public repositories and their history; no rule for which repositories or how many points in time.
Limitation	One snapshot per repository gives only one ranking per project (about 25 queries in total), which is very little data for a ranking model. Unfiltered repositories include ones with no issue tracker, too little history or mostly vendored code, which produce empty or misleading labels.
Change	Selection criteria (at least 3 years and 1,000 commits, 80 or more source files, bug-labelled issues, recent activity, permissive license, diverse domains) and snapshots every 6 months. Rows are (repository, snapshot, file).
Why it is better	About 150 queries and 45,000 rows from 25 repositories, all grounded in real history, and labels that exist because the repositories actually track bugs.
Cost / trade-off	Rows from the same repository are correlated; this is handled by splitting by repository and time (C11).
One-line defence	*"We sample each project at several points in time and only use projects whose bug tracking makes the labels meaningful."*

C5 Measure the quality of bug-fix detection

Fixes a flaw | Medium

Original plan	Bug/fix history is an input and drives the labels, but how bug fixes are detected is not specified.
Problem it causes	Keyword matching alone has well-known false positives and misses fixes with plain messages. If label quality is unknown, the reported model quality is unknown too.
Change	Three-tier detection (issue links with bug labels, filtered keywords, an optional message classifier) and a manual check of 200 sampled commits, reporting precision and recall.
Why this solves it	Label noise becomes a measured number in the report instead of a hidden assumption.
Cost / trade-off	A few hours of manual labelling.
One-line defence	*"We hand-checked a sample of commits, so we know how accurate our bug labels are."*

4. Learning Objective and Scoring

C6 Learning-to-rank (LambdaMART) instead of plain classification	Fixes a flaw | High
Original plan	Train Random Forest or XGBoost on the labels (classification or regression).
Problem it causes	The product shows a ranked list and users act on its top. Classification and regression losses treat every file equally, so the model spends its effort on the hundreds of unremarkable files in the middle and is not rewarded for getting the top 10 right. The objective does not match how the output is used.
Change	Train an XGBoost LambdaMART ranker (objective rank:ndcg) with one query per repository-snapshot. Keep Random Forest and a pointwise XGBoost model as baselines.
Why this solves it	LambdaMART weights each training pair by how much swapping it would change NDCG, so mistakes at the top of the list cost the most. The model is trained for exactly what the user sees. It is still XGBoost and still works with TreeSHAP, so nothing else in the stack changes.
Cost / trade-off	Ranking scores have no units, which C7 solves.
One-line defence	*"Users act on the top of a list, so we train with a ranking loss that cares most about the top of the list."*

C7 Give the 0 to 100 score a defined meaning	Fixes a flaw | High
Original plan	"Technical Debt Priority Score: 92/100 (Critical)", with no definition of the scale or the bands.
Problem it causes	A number like 92 invites the question "92 of what?". If it is a raw model output squashed to 0-100, it changes between model versions, is not comparable across repositories, and the Critical threshold is arbitrary. A score that cannot be explained undermines trust in the whole tool.
Change	Report two numbers. **Priority** is the file's percentile within its repository (92 means it ranks above 92% of files), with fixed bands (Critical 90 or above, High 75 to 89, Medium 50 to 74, Low below 50). **Risk** is a calibrated probability of a latent defect being fixed in the next 12 months, from isotonic regression, checked with a reliability diagram, Brier score and calibration error.
Why this solves it	Both numbers have plain-language meanings, both are stable and testable, and together they answer the two questions users have: "what do I fix first here?" and "how worried should I be?".
Cost / trade-off	Calibration needs a held-out fold. Percentile bands mean every repository has some Critical files, which is why the absolute risk is shown alongside.
One-line defence	*"92 means this file ranks above 92% of the files in its repository, and separately we say it has, for example, a 64% chance of a bug fix within a year."*

C8 Compute the fix order with return on effort and a budget	Fixes a flaw | High
Original plan	Output includes a "recommended fix order", but no method for producing it beyond the score.
Problem it causes	Sorting by risk recommends the biggest, most tangled files first. They are risky, but fixing one may take a whole sprint, and several cheaper files would remove more risk in the same time. A pure risk ranking also tends to favour large files simply because size correlates with defects.
Change	For every file: benefit = calibrated risk x impact (log of transitive dependents); cost = a transparent SQALE-style remediation estimate in hours. A 0/1 knapsack (exact dynamic programming) picks the set with the largest total benefit within the team's budget (for example 40 hours).
Why this solves it	Prioritisation becomes an actual optimisation with a stated objective, and the result answers the real planning question: "what should we do with the time we have?". It is also the clearest algorithmic contribution of the project.
Cost / trade-off	Cost estimates are heuristics. They are shown as ranges, every constant is configurable, and the plan updates instantly when they change.
One-line defence	*"We do not just say what is risky; we pick the fixes that remove the most risk per hour within your budget."*

C9 Order the plan by dependencies	Improvement
Original plan	A fix order, implicitly the score order.
Limitation	Refactoring a low-level module after the modules that depend on it means touching those dependents twice. Files in a dependency cycle cannot be fixed one at a time.
Change	Take the chosen files, collapse dependency cycles into single steps (Tarjan's strongly connected components), sort topologically so dependencies come first, and break ties by return on effort.
Why it is better	The plan is something a team can follow step by step, and each step comes with a reason ("fix before invoice.py, which depends on it").
Cost / trade-off	Negligible; graph sizes are small.
One-line defence	*"Foundations first: the plan never asks you to refactor something before the module it is built on."*

C10 Add strong baselines; keep Random Forest as one of them	Fixes a flaw | High
Original plan	Train Random Forest/XGBoost; no comparison point.
Problem it causes	Without baselines, any score looks like success. Defect-prediction research has shown that simple rules (largest files first, or churn x complexity hotspots) are hard to beat. If the model does not beat them, the "AI" adds nothing, and a reviewer who knows this literature will ask.
Change	Every result is reported against random, LOC, total complexity, the churn x complexity hotspot rule, prior bugs, a Random Forest classifier and a pointwise XGBoost regressor.
Why this solves it	The model's value is demonstrated rather than assumed. The hotspot rule is the industry practice (as in tools inspired by Tornhill's work), so beating it is a meaningful claim.
Cost / trade-off	More experiments to run, all cheap.
One-line defence	*"We compare against the rules practitioners already use, including churn times complexity, and show the margin."*

5. Evaluation

C11 Split by time and by project, never by random row	Fixes a flaw | Critical
Original plan	No evaluation protocol stated; the default in most tutorials is a random train/test split of rows.
Problem it causes	A random split puts files from the same repository and period in both training and test sets. They share authors, history and bursts of activity, so the model recognises the project rather than learning general patterns. Metrics come out inflated and do not hold on a new repository, which is exactly how the tool is used.
Change	Two protocols: a temporal split within projects (train on early snapshots, test on the latest, with an embargo), and leave-one-project-out (train on all repositories but one, test on that one, for every repository). Tuning uses time-ordered, grouped folds.
Why this solves it	Leave-one-project-out measures what users experience (running TDP on a repository the model has never seen); the temporal split measures what happens when a team uses it over time on their own project.
Cost / trade-off	Lower, more variable numbers; more training runs (one per held-out project).
One-line defence	*"Every test repository is one the model never saw during training, and every test period comes after the training period."*

C12 Use ranking and effort-aware metrics	Fixes a flaw | High
Original plan	No metrics stated; accuracy, F1 or AUC are the usual defaults.
Problem it causes	Most files have no future defect, so accuracy rewards predicting "clean" for everything. AUC and F1 ignore position and ignore effort: a model that flags large files looks good, but reviewing them uses the whole budget.
Change	Primary metrics: NDCG@10 and Precision@10 (is the top right?), Recall@20% of LOC and P_{opt} (how much risk is found per amount of code read), Spearman correlation, and Brier score and calibration error for the risk probability.
Why this solves it	These metrics measure the product's actual promise. Effort-aware metrics are standard in defect-prediction research precisely because of the large-file bias.
Cost / trade-off	None beyond implementation.
One-line defence	*"We measure how much real risk you find per line of code you read, not just whether labels match."*

C13 Prove the novelty claim with an ablation study

Fixes a flaw | High

Original plan	Novelty claim: code metrics + repository evolution + dependency impact lead to better prioritisation. No plan to test it.
Problem it causes	An unproven novelty claim is the easiest point to attack in a review. It is also possible that one family of features does all the work and the others add nothing; without an ablation, nobody would know.
Change	Train on static features only, then add evolution, ownership, impact and test/SATD families one at a time; also remove each family from the full model. Report the change in every metric with confidence intervals.
Why this solves it	The central claim of the project becomes a measured result with a chart. If a family does not help, that is reported too, which is still a valid finding.
Cost / trade-off	About ten extra training runs per protocol.
One-line defence	*"Our central claim is tested directly: here is how much each kind of signal adds."*

C14 Report significance and confidence intervals

Improvement

Original plan	Not mentioned.
Limitation	With 20 to 30 repositories, differences between methods can be noise. A single averaged number hides that.
Change	Wilcoxon signed-rank tests across repositories (Holm-corrected), Cliff's delta effect sizes, and 95% bootstrap confidence intervals for every headline number.
Why it is better	Claims like "better than the hotspot rule" come with evidence that the difference is real and how large it is.
Cost / trade-off	Small; standard library functions.
One-line defence	*"We show the improvement is statistically significant across projects, with effect sizes."*

C15 Use PROMISE as a sanity check, not training data

Fixes a flaw | Medium

Original plan	"Supplement it with established software-engineering datasets such as PROMISE."
Problem it causes	PROMISE defect datasets use different metrics (mostly CK object-oriented metrics on Java classes), different label definitions and no repository history. Merging them would mix incompatible features and labels, and they cannot supply the evolution or dependency features the project is about.
Change	Evaluate only the static-feature part of the pipeline on PROMISE as a sanity check: results should fall in the range the literature reports. Train only on the self-built dataset.
Why this solves it	PROMISE still adds value (it catches bugs in the metric extractors and connects the work to prior research) without contaminating the dataset.
Cost / trade-off	None.
One-line defence	*"PROMISE has a different schema and no history, so we use it to validate our extractors, not to train."*

6. Features

C16 Add co-change coupling — Improvement

Original plan	Dependencies from the import graph (dependency count, number of dependents).
Limitation	Import graphs miss real dependencies: dynamic imports, plugin registries, configuration, templates, SQL migrations and code in other languages. Two files that always change together are coupled even if neither imports the other.
Change	Build a co-change graph from git (support and confidence of pairs of files changed in the same commit, excluding bulk commits) and add features such as strong partners and hidden coupling (strong partners not linked by imports).
Why it is better	Captures architecture debt that no static tool can see, is cheap to compute with one sparse matrix product, and is a known strong signal in mining-software-repositories research.
Cost / trade-off	Needs a bulk-commit filter so formatting runs do not create fake coupling.
One-line defence	*"Files that keep changing together are coupled whether or not the import graph says so; we measure that."*

C17 Ownership features instead of a contributor count — Improvement

Original plan	"Number of contributors."
Limitation	A raw count is ambiguous: 10 contributors with one clear owner is healthy; 10 contributors who each made a few changes is not. The count cannot tell these apart.
Change	Owner share, number of minor contributors (under 5% of changes), author entropy, and whether the main owner is still active.
Why it is better	Research on ownership (Bird et al.) found minor contributors predict defects better than the raw count. Owner activity also detects knowledge debt when the person who understood a file has left.
Cost / trade-off	Requires author normalisation (.mailmap), which is a small amount of work.
One-line defence	*"It is not how many people touched a file that matters, but whether anyone really owns it."*

C18 Recency-weighted churn and trend features — Improvement

Original plan	"Git churn" and "commit frequency" over the whole history.
Limitation	Lifetime totals treat a burst of changes from five years ago the same as last month's. Old, settled files look as risky as files under active development.
Change	Exponentially decayed churn and commit counts (default half-life 90 days, tuned), windowed counts, relative churn (churn per line), a complexity trend over the last 10 revisions, and burstiness.
Why it is better	Risk follows current activity. Relative churn is a stronger predictor than absolute churn (Nagappan and Ball), and the trend shows whether debt is growing.
Cost / trade-off	One extra hyperparameter (the half-life).
One-line defence	*"Recent change counts more than ancient change, and we track whether complexity is rising."*

C19 Graph centrality instead of a simple dependents count — Improvement

Original plan	"Dependency count" and "number of dependents" (direct).
Limitation	Direct counts miss reach: a module used by one core module that everything else uses is more critical than its count suggests. Cycles are not visible at all.
Change	Transitive dependents (reverse reachability), PageRank, betweenness centrality, and the size of the dependency cycle containing the file.
Why it is better	Measures the real blast radius of a change and exposes architecture debt (cycles, bridge modules). This is also what makes a claim like "affects 23 files" correct, since it counts indirect dependents.
Cost / trade-off	Betweenness is slower on large graphs; approximate (sampled) betweenness is used above a few thousand nodes.
One-line defence	*"We count everything a change can break, directly or indirectly, not just direct imports."*

C20 Static test signals instead of measured coverage — Fixes a flaw | High

Original plan	"Test coverage" as a feature for every file in every repository.
Problem it causes	Measured coverage requires installing dependencies, building and running each project's test suite at every snapshot. For 25 repositories and 150 snapshots that is impractical, old snapshots often no longer build, and it means executing untrusted code from the internet. The feature would be missing for most rows.
Change	Static proxies: whether a matching test file exists, how many test files import the module, how often changes to the file also touch tests, and the package's test-to-code ratio. Real coverage (coverage.py, in a sandbox) only for two or three demo repositories, where it also validates the proxies.
Why this solves it	The test signal is available for every row, costs seconds, needs no code execution, and its relationship to real coverage is measured rather than assumed.
Cost / trade-off	Proxies are less precise than line coverage; the validation on demo repositories quantifies the gap.
One-line defence	*"Running 150 test suites is infeasible and unsafe, so we use static test signals and validate them against real coverage."*

C21 Normalise features within each repository

Fixes a flaw | Medium

Original plan	Raw metric values pooled across repositories.
Problem it causes	Scales differ by orders of magnitude between projects. Forty commits a month is heavy for a small library but quiet for a big framework. Raw values make the model learn which project a file belongs to, and it transfers badly to new repositories.
Change	log1p for heavy-tailed counts, plus percentile-rank versions of every feature within its repository-snapshot.
Why this solves it	Relative features ("top 5% churn in this repository") mean the same thing everywhere, which is what cross-project use needs. The reasons shown to users already use percentiles, so the explanations get clearer too.
Cost / trade-off	Twice as many columns; trees handle this easily.
One-line defence	*"We describe each file relative to its own project, so the model works on projects of any size."*

C22 Add self-admitted technical debt (SATD)

Improvement

Original plan	Not in the original inputs.
Limitation	Developers often write down debt in comments ("TODO: hack until the API is fixed"). Ignoring this misses the most direct evidence available, and the project had no natural-language component despite the "AI" framing.
Change	Extract comments with Tree-sitter, classify them with TF-IDF and logistic regression trained on a public SATD dataset (about 62,000 labelled comments), and add SATD count, density and age as features and as a debt type.
Why it is better	A cheap, well-studied NLP component that adds signal and directly surfaces the debt developers already know about.
Cost / trade-off	The public dataset is Java; a few hundred hand-labelled Python comments are added for adaptation.
One-line defence	*"When developers admit debt in a comment, we read it."*

C23 Specify the duplication algorithm

Improvement

Original plan	"Duplication" listed as an input without a method.
Limitation	Duplication can mean exact line matches, which miss copy-paste-rename clones, or expensive AST comparison. Without a method the metric cannot be reproduced.
Change	Token-based clone detection: normalise identifiers and literals, hash 50-token windows with Rabin-Karp, keep fingerprints by winnowing, group matches across the repository.
Why it is better	Detects renamed clones, runs in linear time, and the winnowing guarantee makes the detection threshold precise.
Cost / trade-off	Implementation time; small.
One-line defence	*"Duplication uses a standard fingerprinting algorithm that catches renamed copies too."*

7. Explainability and the AI Layer

C24 Group SHAP values by feature family and turn them into sentences	Fixes a flaw | Medium
Original plan	SHAP is in the stack to provide "top reasons".
Problem it causes	Raw SHAP output lists dozens of features with signed decimals, which developers cannot act on. With correlated features (lines of code, total complexity, number of functions) the credit is split unpredictably among them, so the "top reason" can change between similar files or model versions.
Change	Sum SHAP values within each feature family, choose the top families, then fill readable templates with the raw value and its percentile ("changes very often: 47 commits in 6 months, top 4% in repo"). Show large negative contributions as mitigating factors.
Why this solves it	Family totals are stable even when individual features share credit, and the sentences are actionable. The explanations still add up exactly to the model's output, so nothing is invented.
Cost / trade-off	Templates must be written once per feature.
One-line defence	*"Our reasons are exact SHAP contributions, grouped so they stay stable and written so a developer can act on them."*

C25 Assign a debt type and a remedy	Improvement
Original plan	A score and reasons.
Limitation	A score says how urgent but not what kind of work is needed. "Critical" alone does not tell a developer whether to add tests, split functions or break a dependency cycle.
Change	Assign one or two debt types (complexity, duplication, test, architecture, knowledge, admitted, defect hotspot) from the dominant SHAP families and simple thresholds, each with a typical remedy. Optionally cluster files in SHAP space to discover debt archetypes.
Why it is better	Turns a ranking into a to-do list and feeds both the cost model and the LLM prompt.
Cost / trade-off	Rules need sensible thresholds; they are documented and adjustable.
One-line defence	*"Every flagged file comes with what kind of debt it is and what fixing it involves."*

C26 A grounded LLM advisor for the top files — Improvement

Original plan	"AI-based prioritisation" in the title; the only learning component is a tree ensemble.
Limitation	The recommendations would be generic ("refactor this file"), and the AI framing rests entirely on gradient boosting. Using an LLM naively would create a different problem: confident advice about functions that do not exist.
Change	On request, for a top-ranked file, send an LLM the reasons, the debt type and the most complex functions (excerpt only) and require a JSON plan. Validate the schema, check every referenced function exists in the syntax tree, clip effort to the cost model's range, and label the result as advice. Evaluate it with a rubric and a hallucination rate.
Why it is better	Adds concrete, file-specific next steps where they are most useful, and the validation layer keeps it trustworthy. The ranker stays deterministic and reproducible; the LLM only explains how to fix.
Cost / trade-off	API cost (limited by caching and on-demand use) and privacy: off by default for private code.
One-line defence	*"The model decides what to fix; the LLM suggests how, and we verify every function it mentions exists."*

C27 A what-if simulator, honestly framed — Improvement

Original plan	Not in the original plan.
Limitation	Users cannot see what would change their priorities, which makes the score feel like a verdict rather than a guide.
Change	Let users toggle actionable features (add tests, split the longest function, remove duplication) and see the new priority, risk and plan. Label it clearly as a model counterfactual, not a causal estimate.
Why it is better	Makes the model's behaviour tangible and motivates specific fixes, while the framing avoids overclaiming.
Cost / trade-off	Must not be presented as proof that a fix reduces bugs.
One-line defence	*"You can see how the model would rate a file after a fix; we are explicit that this is not a causal claim."*

C28 Collect feedback and learn from it — Improvement

Original plan	Not in the original plan.
Limitation	A global model cannot know that a team considers a module stable or about to be deleted. Without feedback there is also no way to measure whether users agree with the rankings.
Change	Agree/disagree buttons store pairwise preferences, used for a per-repository adjustment and for active learning (asking about the most uncertain files). Agreement on the top 10 is tracked per model version.
Why it is better	The tool adapts to each team, and user agreement becomes an evaluation signal beyond historical labels.
Cost / trade-off	Needs a few users to be meaningful; the infrastructure is cheap.
One-line defence	*"Teams can correct the ranking, and we measure how often they agree with it."*

8. Scope

C29 File level first; function level later	Scope decision
Original plan	"For each file/function, extract ..."
Problem it causes	Function-level history requires tracking functions across renames, moves and splits, which is a research problem of its own. Attempting both levels at once risks finishing neither, and import graphs and ownership are naturally file-level.
Change	Build and evaluate everything at file level. Function-level complexity is still used as features (maximum and mean per file) and to pick the code shown to the LLM. Function-level ranking is listed as an extension.
Why this solves it	One level done properly, with clean labels and evaluation, is stronger than two done partially.
Cost / trade-off	Coarser recommendations; mitigated by pointing to the most complex functions within each file.
One-line defence	*"We chose the granularity where history, imports and ownership are reliable, and use function metrics inside it."*

C30 Python first, Java second	Scope decision
Original plan	Tree-sitter implies any language.
Problem it causes	Tree-sitter gives a parser per language, but each language still needs its own queries, import resolution and test-file conventions. Supporting many languages thinly produces unreliable features everywhere.
Change	A language plug-in interface; Python is implemented and evaluated first, Java is the next plug-in.
Why this solves it	Results are reliable for the languages supported, and the plug-in design shows how to extend it.
Cost / trade-off	Fewer languages at the demo.
One-line defence	*"We support each language properly or not at all, and the plug-in interface makes the next one straightforward."*

9. Engineering and Platform

C31 Background jobs with progress instead of synchronous requests	Fixes a flaw | High
Original plan	FastAPI backend; no mention of how long-running analysis is executed.
Problem it causes	Cloning and mining a medium repository takes minutes. A synchronous HTTP request will time out at proxies and load balancers, a page refresh loses the work, and one large analysis blocks the server for everyone else.
Change	The API validates and enqueues; Celery workers run the analysis as a chain of retryable stages; progress is streamed to the browser with Server-Sent Events. Separate queues keep LLM calls from waiting behind analyses.
Why this solves it	Long work survives refreshes and worker crashes (late acknowledgement and retries), and the API stays responsive.
Cost / trade-off	Two more services (Redis and workers) to run, all inside Docker Compose.
One-line defence	*"Analysis takes minutes, so it runs as a background job with live progress, never inside an HTTP request."*

C32 PostgreSQL required, not optional	Fixes a flaw | Medium
Original plan	"Optional: PostgreSQL."
Problem it causes	Without a database, results disappear on restart, the same commit is re-analysed every time, trends over time are impossible, and feedback has nowhere to live.
Change	PostgreSQL is the source of truth: analyses keyed by (repository, commit, model version) for idempotency, results with indexed sort columns and JSONB feature vectors, plans, advice, feedback, and the static-metrics cache.
Why this solves it	Enables caching, trends, feedback, reproducibility (every score records its model version) and fast dashboard queries.
Cost / trade-off	One more container; migrations with Alembic.
One-line defence	*"Trends, caching, feedback and reproducibility all need persistent storage, so the database is part of the design."*

C33 Incremental analysis with content-addressed caching	Improvement
Original plan	Each analysis starts from scratch.
Limitation	Re-analysing after a few commits would repeat minutes of parsing and mining, which makes the PR integration (C36) too slow to be useful.
Change	Keep bare mirrors and parse only new commits; cache static metrics by (blob hash, extractor version), so unchanged files are never parsed again.
Why it is better	Re-analysis drops from minutes to seconds, and cache invalidation is automatic because git hashes content.
Cost / trade-off	Disk space for mirrors; they are a cache and can be deleted at any time.
One-line defence	*"Git already names file contents by hash, so we never parse the same content twice."*

C34 One shared core package for training and serving	Fixes a flaw | Medium
Original plan	Separate pieces (ML notebooks, a FastAPI backend) with no stated link between them.
Problem it causes	When features are computed one way in training notebooks and re-implemented in the backend, small differences creep in and the deployed model silently receives different inputs than it was trained on (training/serving skew).
Change	All analysis and feature code lives in one package used by training, workers, CLI and the GitHub Action, with a test that the two paths produce identical features.
Why this solves it	Removes a whole class of silent production bugs and keeps the codebase smaller.
Cost / trade-off	Requires discipline about package boundaries (the core never imports web frameworks).
One-line defence	*"The exact same code computes features for training and for users, and a test enforces it."*

C35 Model versioning, promotion gates and no pickle files	Improvement
Original plan	Train a model; no plan for versioning or deployment.
Limitation	Without versions, no score can be traced to the model that produced it, and a worse model can replace a better one unnoticed. Pickled models can execute code when loaded.
Change	MLflow tracking and registry; a model bundle of JSON/YAML files (XGBoost JSON, calibrator arrays, feature schema, templates, metadata); automatic promotion gates (beats the hotspot baseline, no regression in NDCG, calibration error at most 0.05, leakage tests pass).
Why it is better	Every result is reproducible and auditable, regressions are blocked automatically, and model files are safe to load.
Cost / trade-off	A small amount of tooling.
One-line defence	*"A model only reaches users after passing the same tests we report, and every score names its model version."*

C36 Bring results into pull requests (GitHub Action and SARIF)	Improvement
Original plan	A Next.js dashboard as the only interface.
Limitation	A separate dashboard is easy to forget. The moment debt matters most is when someone is about to change a risky file.
Change	A GitHub Action that runs inside CI, comments on pull requests touching Critical or High files (one comment, updated in place), and uploads SARIF for GitHub code scanning. A CLI for local use.
Why it is better	Puts the information where decisions are made, needs no hosting for users, and keeps their code inside their CI runner.
Cost / trade-off	Packaging and documentation work.
One-line defence	*"Developers see the risk of the file they are touching in the pull request itself."*

C37 Use a graph library for the dependency view

Fixes a flaw | Low

Original plan	Recharts for the frontend visualisations.
Problem it causes	Recharts covers bars, lines and treemaps but has no node-link graph, so the dependency graph, one of the most useful views for this product, could not be built with it.
Change	Keep Recharts for charts; add Cytoscape.js (or react-force-graph) for the dependency graph.
Why this solves it	Each library does what it is good at.
Cost / trade-off	One more frontend dependency.
One-line defence	*"Recharts cannot draw networks, so the graph view uses a graph library."*

C38 Security hardening for untrusted repositories

Fixes a flaw | Critical

Original plan	Analyse public repositories from a URL; no security considerations.
Problem it causes	A user-supplied URL can point at internal addresses (server-side request forgery). Git hooks, filters and submodules can execute or fetch unexpected content. Huge repositories or pathological files can exhaust disk and CPU. Code comments can carry prompt injections aimed at the LLM.
Change	https-only host allowlist with private-address blocking; bare mirrors with hooks disabled and no submodules; never executing repository code; size, time and file limits; non-root, read-only containers; the LLM gets quoted data, no tools and a strict output schema; secrets server-side only.
Why this solves it	Each control addresses a specific, realistic attack against a service that processes untrusted code.
Cost / trade-off	Some legitimate inputs are refused (very large repositories, unusual hosts) unless configured.
One-line defence	*"We treat every repository as untrusted input: it is parsed, never executed, and every way it can reach our systems is limited."*

C39 Tests, CI/CD and containers from day one

Improvement

Original plan	Not mentioned.
Limitation	Without automated tests and a reproducible environment, results cannot be re-run, regressions in metric code go unnoticed, and a demo depends on one laptop's setup.
Change	Unit tests with known-value fixtures, scripted git repositories for history and SZZ tests, API and end-to-end tests, GitHub Actions CI, Docker images and a Compose file that starts the whole stack with one command.
Why it is better	Correctness of the metric code is checked continuously, and anyone can reproduce the results and the demo.
Cost / trade-off	Setup time in the first week.
One-line defence	*"Everything in the report can be regenerated from a clean clone with one command."*

10. What Was Kept, and What Was Rejected

10.1 Kept from the original plan

Element	Status	Why it stays
Core idea: rank by urgency, not just detect smells	kept	Correct framing; it is what teams need
Self-built dataset from real repositories	kept, specified	Real history is the project's strength; C4 adds criteria and snapshots
Complexity, churn, dependencies, ownership, bug history, duplication	kept, refined	All still used; C16 to C23 sharpen each one
XGBoost	kept	Now as LambdaMART (C6); still fast, tabular-friendly and SHAP-compatible
Random Forest	kept as baseline	Shows what the ranking objective adds (C10)
SHAP	kept	TreeSHAP is exact for trees; C24 makes its output readable
Tree-sitter, GitPython, NetworkX	kept	Right tools for parsing, history and graphs
FastAPI, Next.js, TypeScript, Tailwind, Recharts	kept	Good fit; Recharts joined by a graph library (C37)
PostgreSQL	upgraded to required	See C32
PROMISE	kept, new role	Sanity check instead of training data (C15)

Table 3. Elements kept from the original plan.

10.2 Considered and rejected

Idea	Why it was rejected
Ask an LLM to score files directly	Not reproducible, expensive per file, impossible to calibrate or evaluate against baselines, and open to prompt injection from code. The LLM is used only for advice (C26).
Graph neural networks on the dependency graph	Attractive, but at about 45,000 rows boosted trees on tabular features usually match or beat them, and GNNs are harder to explain. Listed as future work.
End-to-end deep learning on source code	Needs far more labelled data, loses the history signal, and gives no readable reasons.
A Kaggle or PROMISE dataset as the main data	No repository history, so the project's novelty (evolution and impact) could not be tested.
Real coverage for every repository	Infeasible and unsafe at scale (C20).
Many languages from the start	Thin support everywhere instead of reliable support somewhere (C30).
Function-level ranking from the start	Function tracking across renames is its own research problem (C29).
Kubernetes for deployment	Unnecessary for this scale; Compose on one VM, with an optional infrastructure-as-code AWS design, covers the demo and the production-style reference.

Table 4. Ideas considered and rejected.

11. Anticipated Questions and Answers

Q. Isn't using past bugs to predict bugs circular?

It would be if both came from the same period, which is what the original plan risked. Our features stop at time T and our labels count only bugs fixed after T in code that existed at T (C1, C2). Past bugs are legitimately known at prediction time; the model must still predict which files will have new fixes.

Q. Why not just sort by churn times complexity? Tools already do that.

That is one of our baselines (C10). The ablation (C13) and the significance tests (C14) show how much ownership, dependency impact and test signals add on top of it, measured on repositories the model never saw (C11).

Q. Your labels come from SZZ, which is known to be noisy. How can you trust them?

We use the refined variant (ignoring cosmetic lines, bulk commits and commits after the bug report), prefer issue-linked fixes, and report the measured precision of bug-fix detection (C5). Ranking metrics are also less sensitive to label noise than exact counts, and the sensitivity analysis (C3) shows the conclusions are stable.

Q. What does 92 out of 100 actually mean?

The file ranks above 92% of the files in its repository. Separately, we report a calibrated probability, for example 64%, that it has a defect fixed within 12 months (C7).

Q. Why learning-to-rank instead of a normal classifier?

Because users act on the top of the list, and a ranking loss puts the most weight there (C6). The classifier is still reported as a baseline, so the benefit is measured.

Q. How do you know the model works on a new project?

Leave-one-project-out evaluation: every reported number comes from a repository excluded from training (C11).

Q. Isn't the cost estimate made up?

It is a transparent heuristic in the style of SQALE, shown as a range, with every constant configurable (C8). The contribution is the optimisation (best set of fixes within a budget), which works with any cost model, including a team's own estimates.

Q. Where is the AI beyond XGBoost?

A learning-to-rank model, calibrated probabilities, SHAP explanations, an NLP classifier for self-admitted debt (C22), and a grounded LLM advisor with validation (C26). The LLM is deliberately kept out of the scoring so the ranking stays reproducible.

Q. Couldn't a malicious repository attack your server?

Repository code is never executed, URLs are restricted, Git hooks and submodules are disabled, inputs are size and time limited, and LLM output is schema-validated (C38).

Q. What would you do with more time?

Function-level ranking, more languages, survival models for time-to-next-defect, and a user study with a development team comparing their judgement with the rankings.